# Email Enumeration

Email enumeration is the process of discovering valid email addresses tied to an organization — used for phishing campaign planning (in authorized engagements), password spraying attacks, and social engineering recon. It sits alongside subdomain/DNS enumeration as a core part of the OSINT phase, and maps to the same Reconnaissance stage of the Kill Chain and ATT&CK.

---

## 1. Why Email Enumeration Matters

Emails are useful to an attacker (or pentester) for several distinct reasons:

- **Phishing** — the actual target list for a simulated phishing campaign
- **Password spraying** — usernames are often just the email prefix (`o.sawant@company.com` → username `o.sawant`), so harvesting emails effectively harvests valid usernames too
- **Social engineering** — knowing names/roles (HR, IT helpdesk, finance) helps craft convincing pretexts
- **Credential stuffing correlation** — checking harvested emails against known breach dumps (HaveIBeenPwned-style) to see if employees reuse passwords

---

## 2. Guessing Email Naming Conventions

Most organizations follow a predictable pattern. Once you know **one** confirmed employee email, you can usually derive the pattern and apply it to any other known employee name (from LinkedIn, company "About Us" pages, etc.).

**Common patterns:**

```
firstname.lastname@company.com     → omkar.sawant@company.com
firstname@company.com              → omkar@company.com
f.lastname@company.com             → o.sawant@company.com
firstnamelastname@company.com      → omkarsawant@company.com
lastname.firstname@company.com     → sawant.omkar@company.com
firstinitiallastname@company.com   → osawant@company.com
```

**Tools that automate pattern generation from a name list:**

```
# namemash.py - generates common permutations from first/last names
python3 namemash.py names.txt > possible_emails.txt
```

---

## 3. theHarvester — The Core Tool for This

`theHarvester` is a pre-installed Kali tool that pulls emails, subdomains, and employee names from multiple public sources in one pass.

```
theHarvester -d company.com -b google
theHarvester -d company.com -b bing
theHarvester -d company.com -b all           # query all supported sources
theHarvester -d company.com -b google -l 500 # limit results returned
theHarvester -d company.com -b all -f report.html   # save as HTML report
```

**Sources it can pull from (varies by version/API keys configured):**

- Google, Bing, DuckDuckGo search
- LinkedIn
- Hunter.io
- crt.sh (certificate transparency, same as subdomain enum)
- Shodan
- VirusTotal

Some sources need free API keys configured in `theHarvester`'s config file to work at full capacity — worth setting these up once so future scans return richer results.

---

## 4. Hunter.io — Purpose-Built Email Discovery

**Hunter.io** is a dedicated web service (with a free tier and API) specifically for finding and verifying company emails.

```
# Web interface
https://hunter.io/search/company.com

# API usage
curl "https://api.hunter.io/v2/domain-search?domain=company.com&api_key=YOUR_KEY"
```

Returns discovered emails along with:

- **Confidence score** — how certain Hunter is the email is real/active
- **Source** — where the email was found (a specific webpage, LinkedIn, etc.)
- **The naming pattern** it detected for the domain — genuinely useful, since it saves you from manually figuring out the convention

---

## 5. LinkedIn — Manual OSINT for Names/Roles

LinkedIn doesn't give you emails directly, but it's the best source for **names and job titles**, which you then feed into the naming-pattern logic above.

```
site:linkedin.com/in "Company Name" "IT" OR "Security"
```

Manually browsing an org's LinkedIn "People" tab, filtering by department, gives you a solid name list to combine with a confirmed email pattern.

---

## 6. Automated LinkedIn + Pattern Tools

### CrossLinked

```
python3 crosslinked.py -f "{f}.{l}@company.com" company_name
```

Scrapes LinkedIn (via Google/Bing search results, avoiding LinkedIn's own anti-scraping) for employee names at a target company, then applies your specified email format string to generate a full candidate list in one step.

### Snov.io, Skrapp — similar SaaS alternatives to Hunter.io, some with free tiers

## 7. Verifying Email Addresses (Confirming They're Real/Active)

Generated or guessed emails are just guesses until verified. A few approaches, from safest/passive to more active:

### a) SMTP Verification (careful — borderline active technique)

```
# Manually via telnet - checks if the mail server accepts t
he address
telnet mail.company.com 25
HELO test.com
MAIL FROM: <test@test.com>
RCPT TO: <omkar.sawant@company.com>
```

If the server responds `250 OK`, the mailbox likely exists. If `550 No such user`, it doesn't.

**Important caveat:** many modern mail servers deliberately return `250 OK` for *every* address (to prevent exactly this enumeration technique) — so SMTP verification is increasingly unreliable and should never be treated as 100% authoritative.

### b) Automated tools

```
# holehe - checks if an email is registered on various plat
forms (Twitter, Instagram, etc.)
holehe email@example.com

# Hunter.io's built-in verifier
curl "https://api.hunter.io/v2/email-verifier?email=test@co
mpany.com&api_key=YOUR_KEY"
```

### c) Breach database correlation

```
# Check if an email appears in known breaches (indicates i
t's a real, actively used address)
# Via HaveIBeenPwned API (requires API key now)
```

```
curl -s "https://haveibeenpwned.com/api/v3/breachedaccount/
test@company.com" \
  -H "hibp-api-key: YOUR_KEY"
```

## 8. Combining Email Enum With Subdomain/DNS Recon (MX Records)

Before harvesting, checking **MX records** tells you what mail platform an org uses — relevant context for both the enumeration approach and eventual phishing infrastructure planning:

```
dig MX company.com
```

```
company.com.  3600  IN  MX  1  aspmx.l.google.com.   → Google Workspace
company.com.  3600  IN  MX  1  company-com.mail.protection.outlook.com.  → Microsoft 3
65
```

Knowing it's Google Workspace vs. Microsoft 365 matters because each has different login portal URLs, different MFA behaviors, and different known phishing-simulation approaches — this is exactly the kind of detail that shows up in a professional social engineering assessment's planning phase.

## 9. Putting It Together — A Practical Workflow

```
# 1. Find MX records - identify email platform
dig MX company.com

# 2. Harvest emails/names from multiple sources
theHarvester -d company.com -b all -f harvest_report.html

# 3. Cross-check with Hunter.io for pattern confirmation +
confidence scores
curl "https://api.hunter.io/v2/domain-search?domain=compan
y.com&api_key=YOUR_KEY"

# 4. Scrape LinkedIn for additional names not caught by sea
```

```
rch-engine sources
python3 crosslinked.py -f "{f}.{l}@company.com" "Company Na
me"

# 5. Deduplicate and compile final candidate list
cat theharvester_emails.txt hunter_emails.txt crosslinked_e
mails.txt | sort -u > final_emails.txt

# 6. (In an authorized engagement) verify a sample against
breach databases
# to gauge how many are real/active accounts, informing phi
shing campaign realism
```

## 10. Email Enum in the Kill Chain / ATT&CK Context

| Framework | Mapping |
|---|---|
| Cyber Kill Chain | Reconnaissance stage (feeds into Weaponization/Delivery for a phishing campaign) |
| MITRE ATT&CK | T1589.002 (Gather Victim Identity Information: Email Addresses), T1598 (Phishing for Information) |

Harvested emails feed directly into later kill-chain stages — this recon output is literally the target list for the Delivery stage of a phishing-based attack.

## 11. Real-World Case Reference

**RSA SecurID breach (2011)**, which you already studied under the Kill Chain — the phishing email was sent to a small, specific group of RSA employees, not a random blast. That kind of **targeted** phishing (spearphishing) only works because the attacker first did exactly this kind of email/name enumeration to build a precise target list rather than guessing.

## 12. Legal & Ethical Notes

- Passive OSINT (search engines, LinkedIn browsing, crt.sh, theHarvester's passive sources) carries essentially no legal risk — you're aggregating public information

- **SMTP verification is more active** and borderline — repeatedly probing a company's mail server, even just to check address validity, can trigger their email security monitoring and should only be done within an authorized engagement's defined scope
- Actually **sending** phishing emails (even simulated ones) requires explicit written authorization — this is standard practice for social engineering assessments, always scoped and signed off before execution
- Using harvested emails for real, non-consensual phishing is straightforwardly illegal (fraud, unauthorized access laws) — the entire value of this skill for your career is in **authorized** assessments and CTF/lab contexts

---

## Quick Reference Cheat Sheet

```
dig MX company.com                                    # identify
mail platform

theHarvester -d company.com -b all                    # multi-sou
rce email/name harvest
theHarvester -d company.com -b google -l 500

curl "https://api.hunter.io/v2/domain-search?domain=compan
y.com&api_key=KEY"   # Hunter.io lookup

python3 crosslinked.py -f "{f}.{l}@company.com" "Company Na
me"   # LinkedIn scrape + pattern apply

holehe email@example.com                               # check pl
atform registration

curl -s "https://haveibeenpwned.com/api/v3/breachedaccount/
EMAIL" -H "hibp-api-key: KEY"  # breach check
```

---